

MLDS Exploratory Data Analysis & Visualisation

Cluster Analysis & Visualisation

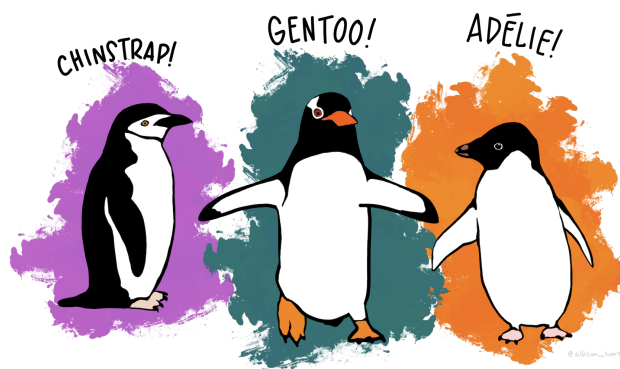
Spring Term, 2025

- Introduction
 - The Palmer Penguin Dataset
- Visualising clusters in one and two dimensions
- Clusters in projected data
- Algorithmic approaches to detecting clusters
 - k -means clustering
- References

Introduction

In this week's reading material, you considered the idea of datasets containing *clusters* of observations, loosely defined as being groups of observations where the within-group variation is lower than the between-group variation. Detection of such data clusters can be both an art and a science. Sometimes, we will be able to discern distinct clusters of observations purely through visualisation of the data; as we will see below, however, visualisations can sometimes be misleading and so we need to be aware of their limitations. Cluster detection can alternatively be addressed using more formal approaches, some of which fall under the umbrella of unsupervised machine learning; we will take a high-level view of one such technique below.

The Palmer Penguin Dataset



We will consider the detection and visualisation of clusters within the 'Palmer Penguins' dataset (Horst, Hill & Gorman, 2020). This data describes the physical dimensions of 344 penguins, which were observed nesting on one of three islands (Biscoe Point, Dream Island and Torgerson Island) of the Palmer Archipelago of Antarctica in 2007, 2008 and 2009. As we will see, this is a particularly good dataset for visualising clusters, as the data refers to three species of penguin (Adélie, Gentoo and Chinstrap), which display (somewhat) distinct physical characteristics.

The specific physical characteristics recorded in the data are the bill length and depth (in mm), as well as the flipper length (in mm) and the body mass (in grams); also included in the data are records of the species and sex of each individual, as well as the year and island on which the observation was recorded.

Our exploratory analysis will focus on the physical dimensions of the penguins; we drop the `island`, `sex` and `year` variables from the dataframe. We then start by considering a summary of the data.

```
library(palmerpenguins)

# first create a smaller tibble, containing only the species and physical characteristics of each penguin
penguin_phys <- penguins %>% select(-c(island,sex,year))

# summarise the data
summary(penguin_phys)
```

```
##      species    bill_length_mm bill_depth_mm flipper_length_mm
## Adelie      :152    Min.      :32.10    Min.      :13.10    Min.      :172.0
## Chinstrap: 68    1st Qu.:39.23    1st Qu.:15.60    1st Qu.:190.0
## Gentoo     :124    Median :44.45    Median :17.30    Median :197.0
##              Mean      :43.92    Mean      :17.15    Mean      :200.9
##              3rd Qu.:48.50    3rd Qu.:18.70    3rd Qu.:213.0
##              Max.      :59.60    Max.      :21.50    Max.      :231.0
##              NA's      :2        NA's      :2        NA's      :2
## body_mass_g
## Min.      :2700
## 1st Qu.:3550
## Median :4050
## Mean      :4202
## 3rd Qu.:4750
## Max.      :6300
## NA's      :2
```

We see that the data contains two observations for which the physical characteristics contain missing values. We further explore by filtering the original `penguins` dataframe, to view only these two observations.

```
penguins %>% filter(is.na(bill_length_mm))
```

```
## # A tibble: 2 × 8
##   species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>          <dbl>          <dbl>          <int>        <int>
## 1 Adelie Torgersen         NA            NA            NA            NA
## 2 Gentoo  Biscoe             NA            NA            NA            NA
## # i 2 more variables: sex <fct>, year <int>
```

In each of the variables for which these two observations contain non-missing values (`species`, `island` and `year`), the observations take two distinct values; there is no evidence that the probability of a value being missing is dependent on these observed values. It is plausible that the missingness could be dependent on the *unobserved* values, however for so few affected observations, even if this is the case, then the impact of any bias caused by misclassifying the mechanism of missingness will be very small. We proceed, assuming these observations to be MCAR and discarding them.

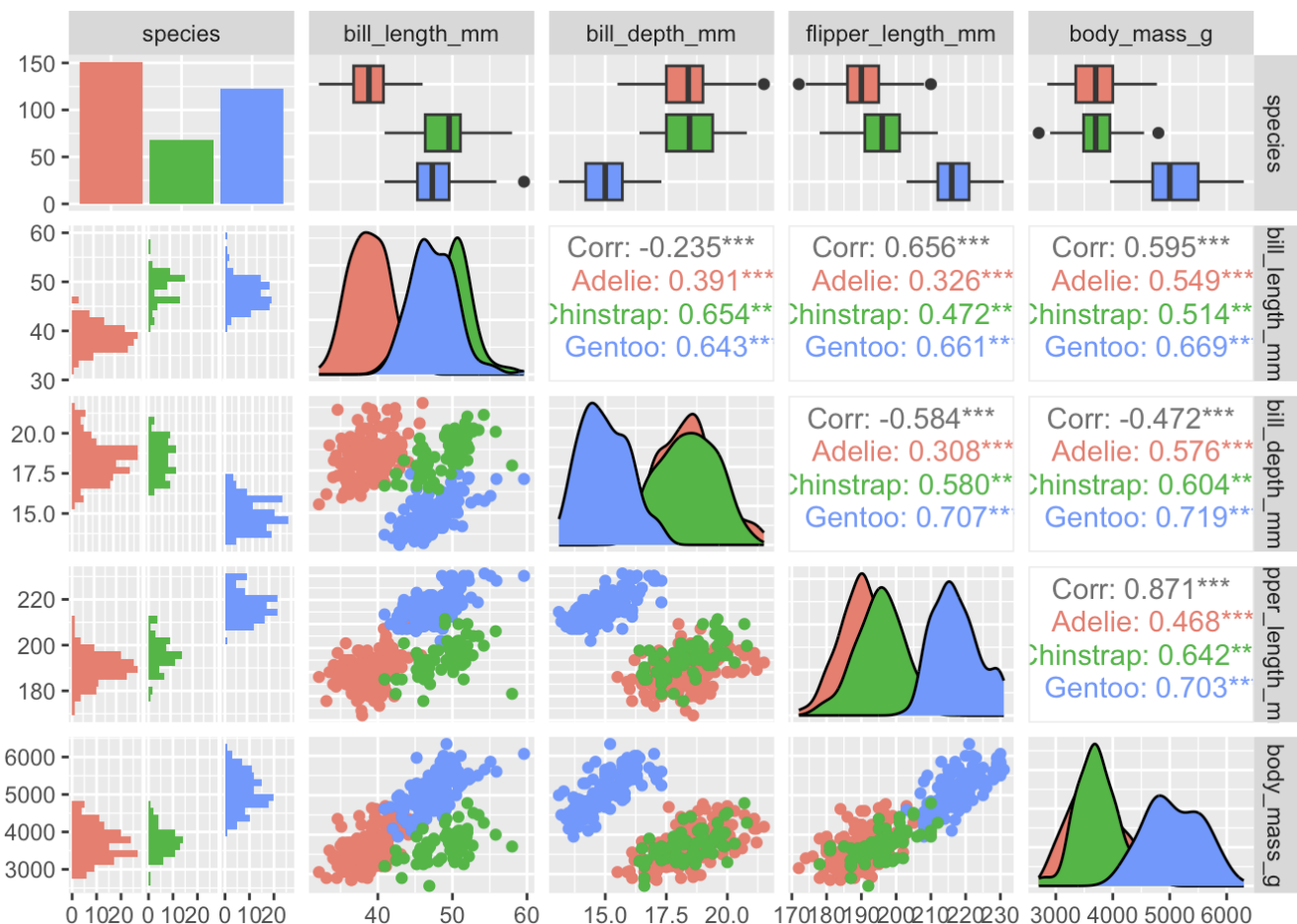
```
penguin_phys <- penguin_phys %>% filter(!is.na(bill_length_mm))
```

We can also visualise this data reasonably straightforwardly, using a generalised pairs plot (provided by

`GGally::ggpairs()`), conditioning on the species of each observation through mapping the colour aesthetic.

```
# explore physical characteristics using ggplots
ggpairs(data = penguin_phys,
        mapping = aes(colour=species),
        progress = FALSE, # suppress verbose progress bar output
        lower = list(combo = wrap('facethist', bins=20))
        )
```

```
## Warning in geom_point(): All aesthetics have length 1, but the data has 25 rows
.
## i Did you mean to use `annotate()`?
```



As we can see, the penguins' physical characteristics are clustered according to their species.

So for this data, we have a variable that intuitively explains the clustering. This will not always be the case though. Suppose we are ignorant of the penguins' species: can we identify the clusters within the data, without *a priori* knowledge of this variable? A further question of interest is whether, for individual observations that are close to the boundary of two clusters, we are able to ascertain which cluster it is more likely to belong to?

Visualising clusters in one and two dimensions

As discussed by Everitt et al (2011, Chapter 2), multimodality in a simple histogram or density estimate

can sometimes indicate the presence of clusters within a single variable.

Suppose that we are ignorant of the species of each penguin; let us drop `species` from the dataset. We might first consider exploring the univariate distributions of each of the physical characteristics - bill length & depth, flipper length and body mass.

```
penguin_nospec <- penguin_phys %>% select(-species)

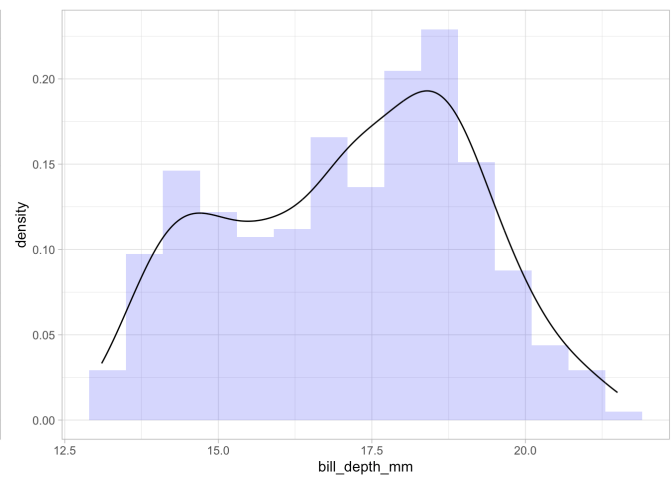
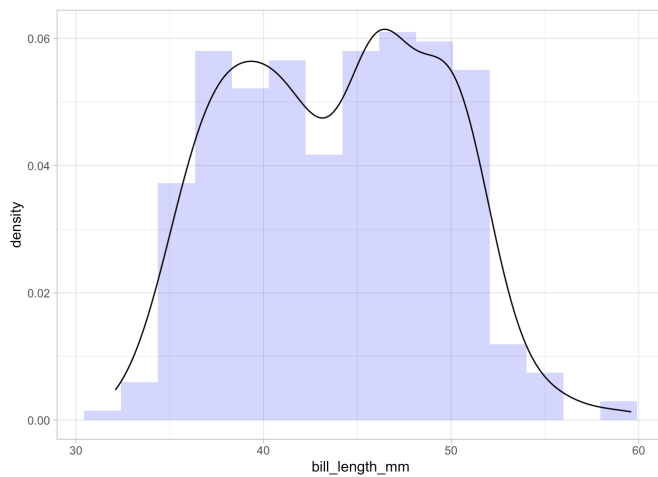
nbins <- 15;

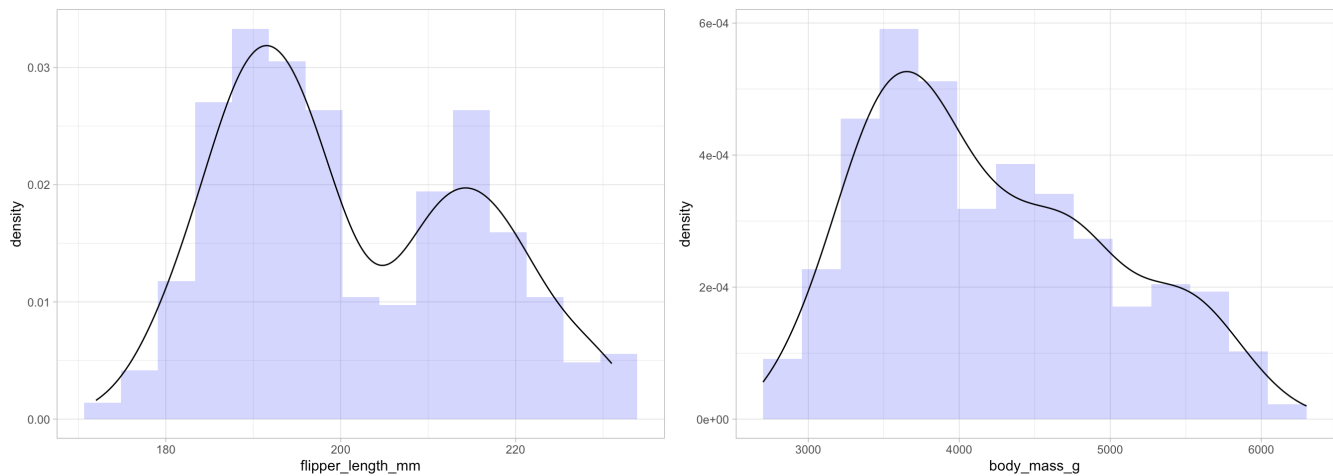
ggplot(data = penguin_nospec, mapping = aes(x=bill_length_mm)) +
  geom_histogram(mapping = aes(y=after_stat(density)),
    bins = nbins, fill="blue", alpha=0.2) +
  geom_density(colour="black") + theme_light()

ggplot(data = penguin_nospec, mapping = aes(x=bill_depth_mm)) +
  geom_histogram(mapping = aes(y=after_stat(density)),
    bins = nbins, fill="blue", alpha=0.2) +
  geom_density(colour="black") + theme_light()

ggplot(data = penguin_nospec, mapping = aes(x=flipper_length_mm)) +
  geom_histogram(mapping = aes(y=after_stat(density)),
    bins = nbins, fill="blue", alpha=0.2) +
  geom_density(colour="black") + theme_light()

ggplot(data = penguin_nospec, mapping = aes(x=body_mass_g)) +
  geom_histogram(mapping = aes(y=after_stat(density)),
    bins = nbins, fill="blue", alpha=0.2) +
  geom_density(colour="black") + theme_light()
```





Exercises

1. Do these visualisations provide clear evidence of the three species-related clusters in the data?

The above histogram and density estimates provide some evidence of multimodality in the marginal distributions, however it's not particularly clear (without *a priori* knowledge of the species variable) that there are three clusters.

2. Can you change any input parameters to `geom_histogram()` and `geom_density()` to obtain clearer indication of these three clusters?

We can, of course, change the resolution of the histogram by specifying the number of bins; this is specified to ggplot2's histogram geom through the input parameter `bins=` (and I've defined the parameter `nbins` above, simply to control this input to all four histograms simultaneously). By increasing the resolution, we might reveal further modes. For instance, setting `nbins=20` will arguably reveal three modes in the first histogram, corresponding to the three species clusters.

We should be careful though! Setting `nbins=15` above might arguably reveal visual evidence of three nodes in the fourth histogram (visualising the marginal distribution of `body_mass_g`). Comparison with the pairs plot above, however, shows that these do not, in fact, correspond to the three clusters. Furthermore, if we continue to increase the number of bins, we can end up with multiple spurious 'modes' in the histogram that don't reflect the true underlying distribution.

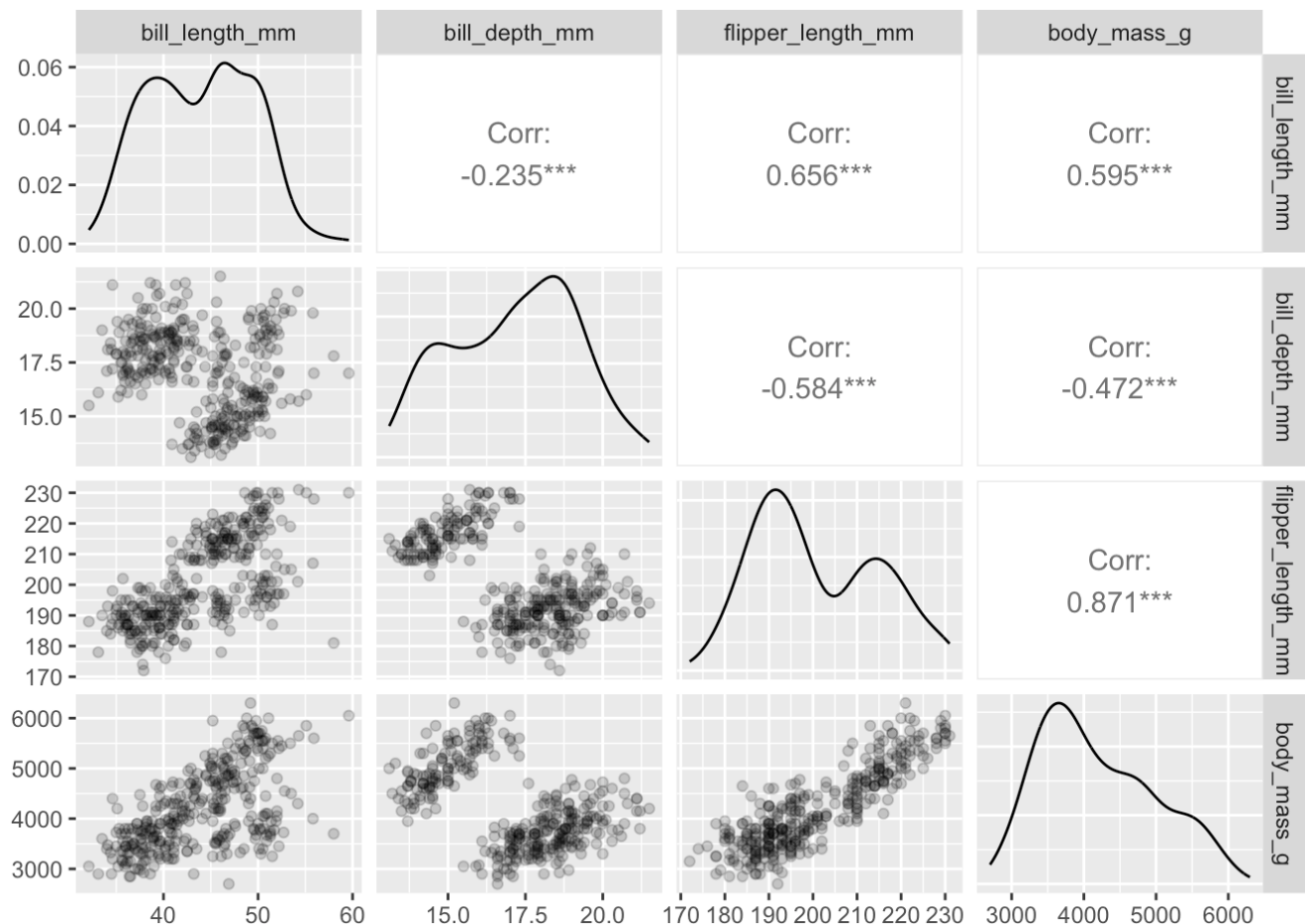
In a similar way, we can change the appearance of the density estimate by altering the bandwidth of the (default Gaussian) kernel that is used for this estimation; we can do so by specifying the input parameter `bw=` for the density geom. As mentioned by Everitt et al (2011, Chapter 2), choosing an appropriate value for the bandwidth is tricky, and is considered in detail by Silverman (1986); the default value used by `geom_density()` obeys a 'rule of thumb' proposed there. More detail on this rule of thumb, as well as alternative rules for choosing the bandwidth, can be found by accessing the help documentation at `?geom_density`

Whilst the histograms and 1D density estimates have presented some evidence of clustering, the above exercises should have demonstrated that it is not necessarily a reliable approach to take. In particular, we have seen a common problem with these approaches: in clustered multivariate data, visualisations of the marginal distributions can misrepresent the number of clusters, as those clusters with similar marginal behaviour will 'mask' one another. For each of the four physical characteristics in our penguin dataset, two out of the three species share a similar enough marginal distribution, such that it is very difficult to discern three distinct clusters using only these one-dimensional visualisations.

We can try to remedy this 'masking' issue by moving to two-dimensional visualisations. Let us repeat the generalised pairs plot for our data, without conditioning on the species variable.

```
ggpairs(data = penguin_nospec,
        progress = FALSE, # suppress verbose progress bar output
        lower = list(continuous = wrap('points', alpha=0.2))
        )
```

```
## Warning in geom_point(): All aesthetics have length 1, but the data has 16 rows
.
## i Did you mean to use `annotate()`?
```



This has helped a little – we can perhaps recognise three clusters in two of the six scatterplots (bill_length_mm vs bill_depth_mm and bill_length_mm vs flipper_length_mm). Unfortunately, however, it has not remedied the issue completely - there are still some clusters masking one another (see, for example bill_depth_mm vs body_mass_g). Furthermore, by comparing (e.g.) the bottom-left panel with the corresponding panel in the previous plot matrix, we can see that, even if clusters do not significantly overlap, their separation can be difficult to establish by eye.

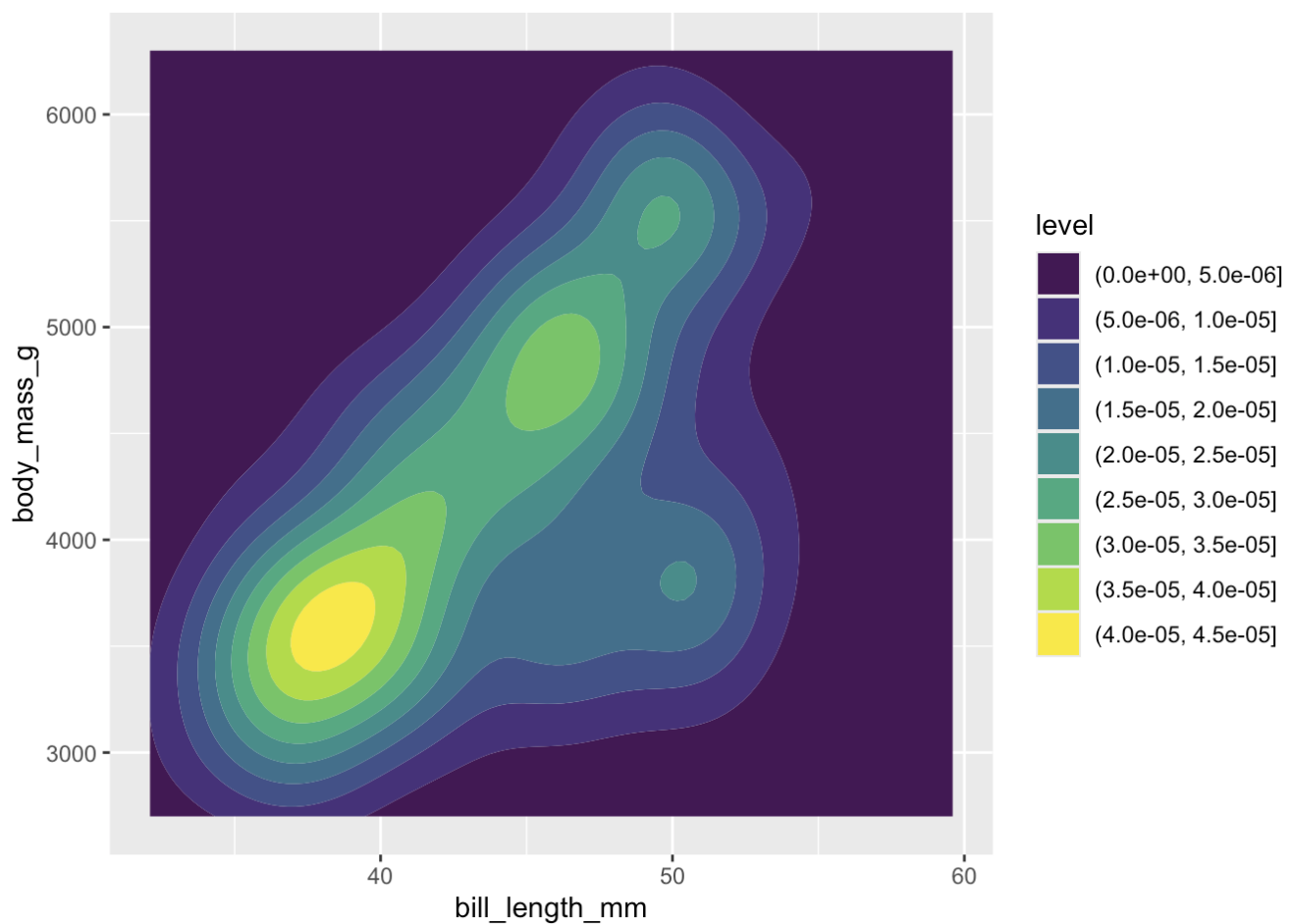
The second of these issues can be helped by using bivariate (kernel-based) density estimation as discussed in Everitt et al (2011, Chapter 2); we can visualise such estimates using contour plots.

Exercises

For the data contained in `penguin_nospec` :

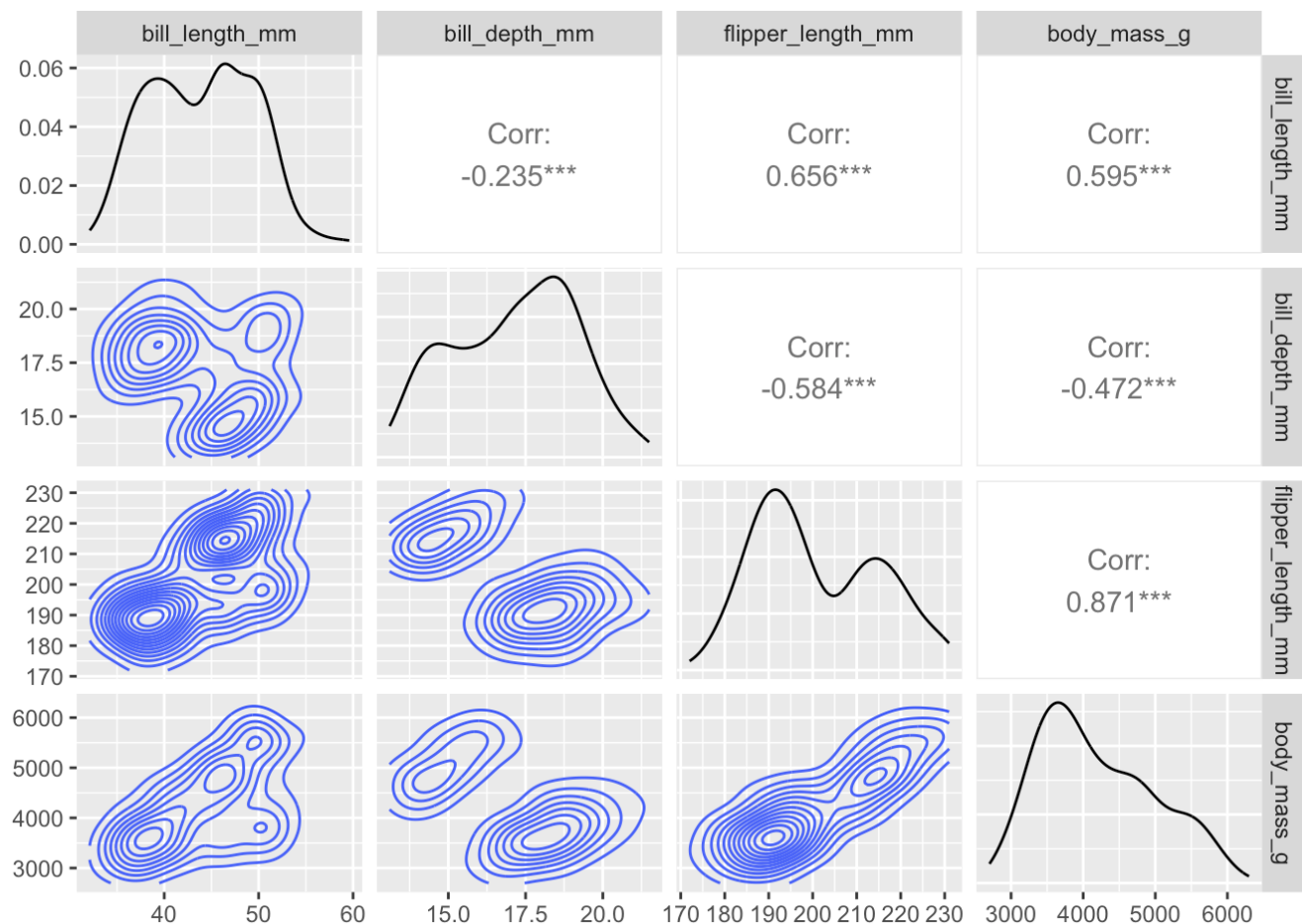
- Using the `ggplot2` library, construct a contour plot that visualises the joint distribution of `bill_length_mm` and `body_mass_g`. *Hint: you may wish to compare and contrast `geom_contour` and `geom_density2d`, as well as `geom_contour_filled()` and `geom_density2d_filled()`.*
- By specifying the input argument `lower=` to `GGally::ggpairs()`, construct a matrix of pairs plots that use contour plots below the diagonal.

```
ggplot(penguin_nospec, aes(x=bill_length_mm, y=body_mass_g)) +  
  geom_density_2d_filled()
```



```
ggpairs(data = penguin_nospec,  
  progress = FALSE, # suppress verbose progress bar output  
  lower = list(continuous = 'density')  
)
```

```
## Warning in geom_point(): All aesthetics have length 1, but the data has 16 rows  
.  
## i Did you mean to use `annotate()`?
```

The output from the above exercises should demonstrate that the bivariate density estimates provide useful extra information, for instance clarifying the separation of clusters in `bill_length_mm` vs `body_mass_g` and in `flipper_length_mm` vs `body_mass_g`. Care should be taken when interpreting the output, however. For instance, when taken in isolation, the bottom-left plot (`bill_length_mm` vs `body_mass_g`) could be interpreted as showing four separate clusters. If we consider this relationship alongside the other bivariate relationships involving `body_mass_g`, we can perhaps convince ourselves that the two small clusters corresponding to larger values of `body_mass_g` are, in fact, part of a single, larger cluster.

Clusters in projected data

As we saw last week, when presented with high-dimensional datasets, it's often useful to project these data onto a low-dimensional embedding. In particular, we saw that projecting onto a two- or three-dimensional embedding allows visualisation, which can sometimes reveal clustering behaviour that is less apparent in high dimensions.

The two-dimensional visualisations that we have discussed above can, of course, be used to combined with dimension reduction to enhance our ability to perform cluster detection through visualisation.

Exercises

In the following exercises, we will perform principal component analysis on the data in `penguin_nospec`.

- Use `prcomp()` to implement PCA on `penguin_nospec`. Extract the first and the second columns of the projected data, and assign them to the variables `pc1` and `pc2`, respectively; place these in a dataframe, `pc_df`, along with the corresponding species labels (from `penguin_phys$species`).
- Calculate the total variance accounted for by the top two principal components.
- Construct a bivariate visualisation of the data's projection onto the top two principal components;

enhance this visualisation with a bivariate density estimate.

8. Would our analysis benefit significantly from also considering the third principal component, i.e. projecting the data onto the top three PCs and using a three-dimensional visualisation?

```
penguin_pca <- prcomp(penguin_nospec, center = TRUE, scale. = TRUE)
```

```
pc1 <- penguin_pca$x[,1]
```

```
pc2 <- penguin_pca$x[,2]
```

```
pc_df <- data.frame(pc1,pc2,species=penguin_phys$species)
```

```
cumsum_values <- cumsum(penguin_pca$sdev^2)
```

```
sum_values <- sum(penguin_pca$sdev^2)
```

```
cat("Cumulative variance across top k PCs, for k=1,...,4:",cumsum_values,"\n")
```

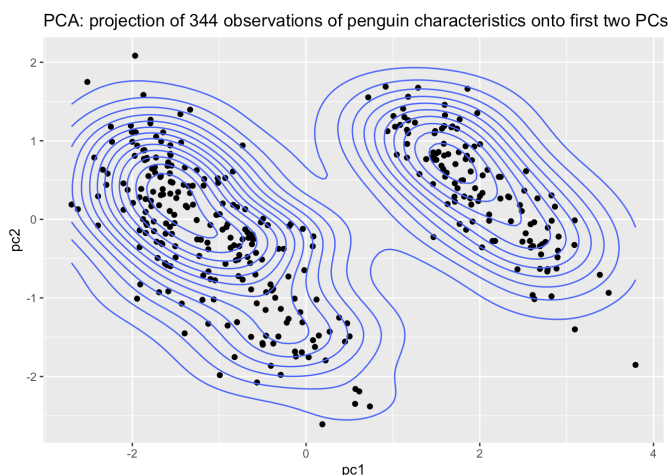
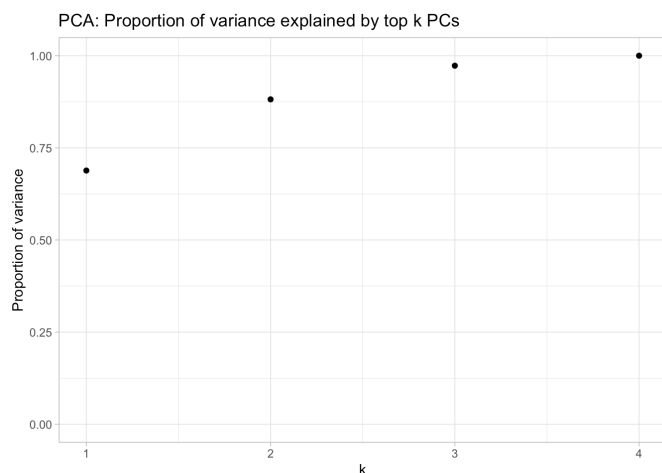
```
## Cumulative variance across top k PCs, for k=1,...,4: 2.753755 3.526272 3.891508
4
```

```
cat("Proportion of variance across top k PCs, for k=1,...,4:",cumsum_values/sum_e
values,"\n")
```

```
## Proportion of variance across top k PCs, for k=1,...,4: 0.6884388 0.881568 0.97
28769 1
```

```
ggplot(data = data.frame(index=1:4,var_prop=cumsum_values/sum_values)) +
  geom_point(mapping = aes(x=index,y=var_prop)) +
  ylim(c(0,1)) +
  ggtitle("PCA: Proportion of variance explained by top k PCs") +
  ylab("Proportion of variance") + xlab("k") + theme_light()
```

```
ggplot(pc_df, aes(x=pc1,y=pc2)) +
  geom_point() +
  geom_density_2d() +
  labs(title="PCA: projection of 344 observations of penguin characteristics onto
first two PCs")
```



In the left-hand panel above, we show the cumulative variance across the four principal components. As detailed in the printout from the above code, we can see that the variance across the top two principal components is equal to 3.53 (2dp), and that this accounts for 88.16% of the total variance. In answer to Q8, we can also see that incorporating the third PC will allow us to account for a combined 97% of the total variance in the original four-dimensional dataset. If we can adequately visualise the resulting projection (in three dimensions), this may be worth exploring; this is left as an independent exercise.

Algorithmic approaches to detecting clusters

We now turn our attention to a more formal approach to cluster detection. We will consider the implementation and visualisation of a cluster detection algorithm, k -means clustering. I provide a high-level overview, before demonstrating its implementation. You will consider formal approaches to clustering in more detail in *Unsupervised Learning*, (Term 4).

k -means clustering

Recall that we loosely defined observation clusters to be groups of observations for which the within-group variation is smaller than the between-group variation. With k -means clustering, we seek to formalise this idea by quantifying the variation within- and between-clusters using the sum of squared (Euclidean) distances. We seek to allocate all observations to one of k clusters, such that the total sum of squared distances (between observations and their allocated cluster centroids) is minimised. The number of clusters, k , is typically provided as an input parameter to any computational implementation of k -means, so that the minimisation of the *within-cluster sum of squares (WCSS)* is with respect to a fixed number of clusters.

We will use the standard implementation of k -means in R, provided by the function `stats::kmeans()`. The two required input arguments for this function are `x` and `centers`: `x` contains the data to be clustered (we can provide this as a dataframe) and `centers` can be either the number of clusters (i.e. k), or a k -length vector of possible cluster centres, from which the minimisation procedure is initialised. For an $n \times p$ dataset, the output from this procedure includes:

- `cluster`, an n -length vector of integers (between 1 and k), indicating the cluster to which each observation belongs;
- `centers`, an $k \times p$ matrix containing the p -dimensional locations of the resulting k cluster centroids;
- `withinss`, a k -length vector containing the within-cluster sum of squares, for each of the k clusters;
- `tot.withinss` the total within-cluster sum of squares, i.e. `sum(withinss)`; and
- `totss`, a scalar value giving the total sum of squares $\sum_{i=1}^n (x_i - \bar{x})^2$ for observations $x_i \in \mathbb{R}^p$, $i = 1, \dots, n$;
- `betweenss`, a scalar value giving the between-cluster sum of squares, which is equal to `totss - tot.withinss`.

In the following code chunk, we demonstrate the application of k -means clustering to the principal components projection of the penguin dataset (using the first two principal components). Since the bivariate density estimate for this principal components projection suggests at least two clusters, without convincing evidence of three, we start by considering $k = 2$.

We can immediately visualise the results of this clustering procedure using the `ggplot2` library, by simply mapping the colour aesthetic to the `kmeans()` output, `cluster`. In the following, we also include the

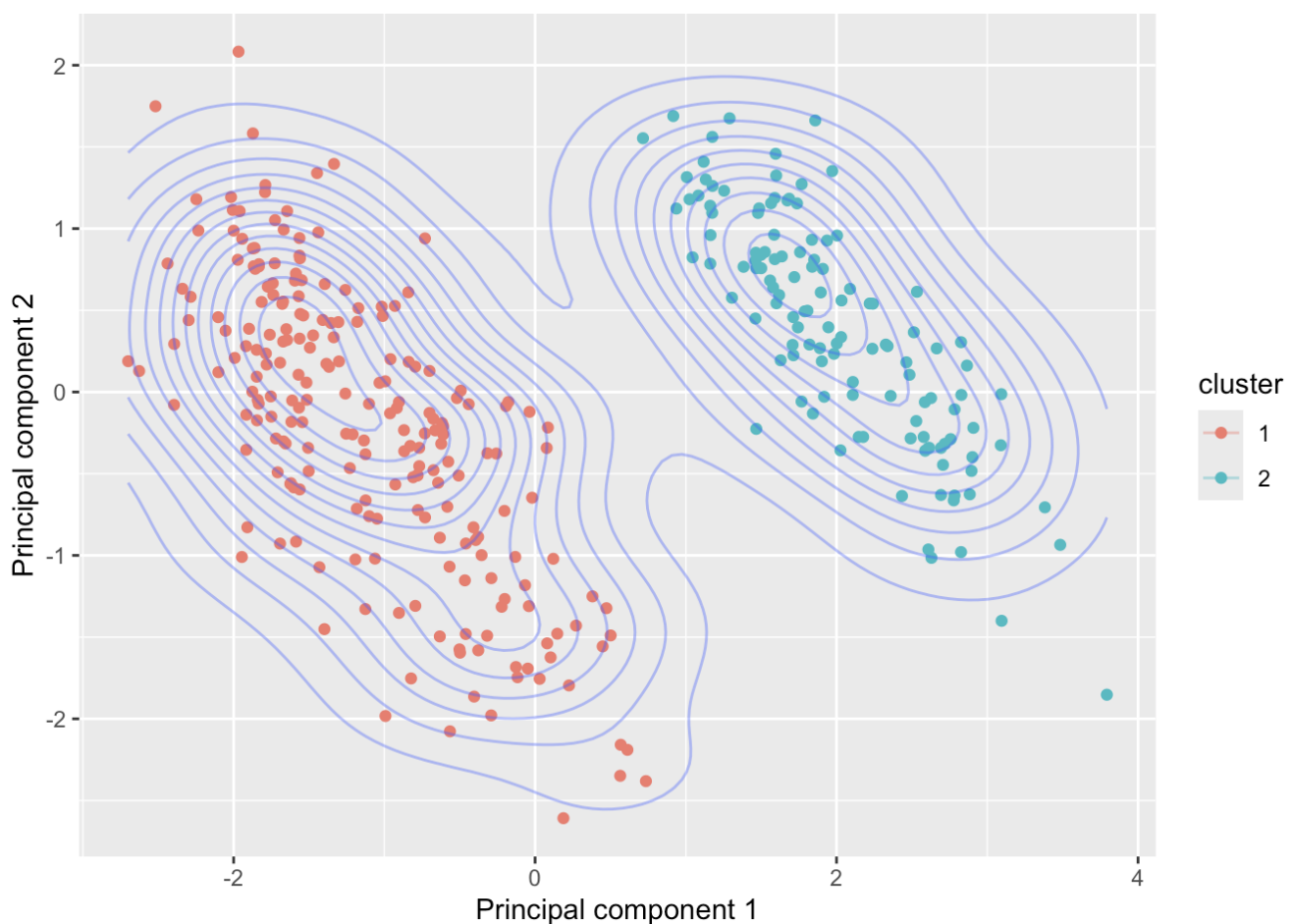
bivariate density estimate of the projected data, for additional context.

The following code depends on the above implementation of PCA; you should uncomment and execute this cell once you have completed Exercises 5-8.

```
penguin_pca_km2 <- kmeans(penguin_pca$x[,1:2], centers=2)

pc_km_df <- bind_cols(pc_df, cluster=as.factor(penguin_pca_km2$cluster))

ggplot(pc_km_df, aes(x=pc1, y=pc2)) +
  geom_point(mapping = aes(colour=cluster)) +
  geom_density_2d(alpha=0.4) +
  xlab("Principal component 1") + ylab("Principal component 2")
```



Exercises

- How can we use the output of `kmeans()` to test whether the principal component projection of `penguin_nospec` does, in fact, contain three clusters?

As detailed above, one of the outputs of `kmeans()` is the variable `tot.withinss`, which gives the total (minimised) within-cluster sum of squares (WCSS) for each value of k . Increasing the number of clusters (i.e. increasing k) will naturally decrease the WCSS, but at additional modelling 'expense'. A sensible choice of k will be such that an additional cluster will no longer be worth the additional expense. This is the intuition behind once more employing the 'elbow' heuristic. We calculate and plot the minimised within-cluster sum of squares for different values of k , and we seek a suitable elbow point.

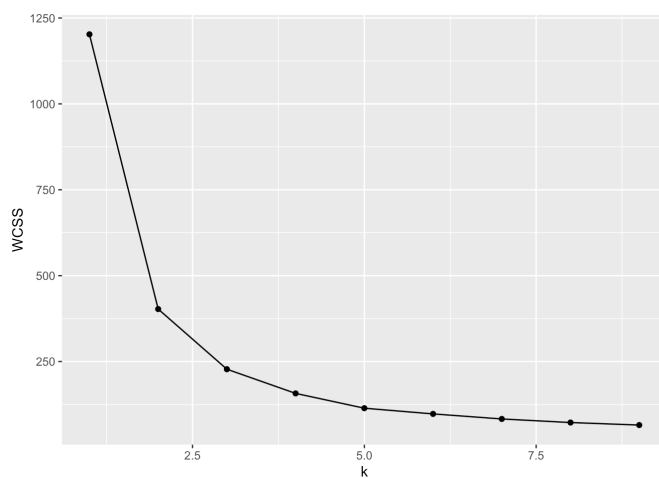
- Implement your suggestion, and interpret your results.

We implement this in the code cell below, finding an elbow at $k = 3$. Note that the clusters (and the minimised value of the WCSS) that are output by `kmeans()` can be sensitive to the initial (random) values

that are allocated as cluster centres. As a result, it is often sensible to run the k -means optimisation procedure multiple times (for each fixed k), each time using a different set of random starting points for the cluster centres. We can achieve this by specifying the input parameter `nstart` to be greater than 1.

```
WCSS_vec <- c()
for(k in 1:9){
  penguin_km_k <- kmeans(penguin_pca$x[,1:2], centers=k, nstart=10)
  WCSS_vec[k] <- penguin_km_k$tot.withinss
}

ggplot(data.frame(k=1:9, WCSS_vec)) +
  geom_line(mapping=aes(x=k, y=WCSS_vec)) + geom_point(mapping=aes(x=k, y=WCSS_vec))
+
  ylab("WCSS")
```



There exist additional packages in R that are designed specifically for visualising the output of `kmeans()` ! In the following, we demonstrate the use of `factoextra::fviz_cluster()` for visualisation of k -means clustering output. In terms of graphic output, it offers a similar visual to the above, but with two key additions: the cluster centres are automatically plotted, and the convex hull of each cluster is also automatically plotted.

`fviz_cluster()` takes the output of `kmeans()` as its first input argument; its second input argument is the $n \times p$ dataset that was originally input to `kmeans()`. The resulting visualisation is a two-dimensional scatterplot of the observations, with the clusters clearly indicated by colour and point shape. If $p > 2$ then PCA is automatically implemented to yield a two-dimensional visualisation of the data.

Exercises

- Using the documentation in `?fviz_cluster` to guide you, visualise the output from the above implementation of k -means ($k = 3$) on the principal components projection of the penguin data.

As you will have noticed, the above implementation of k -means actually used $k = 2$ - we therefore repeat the implementation for $k = 3$ and visualise using `fviz_cluster`.

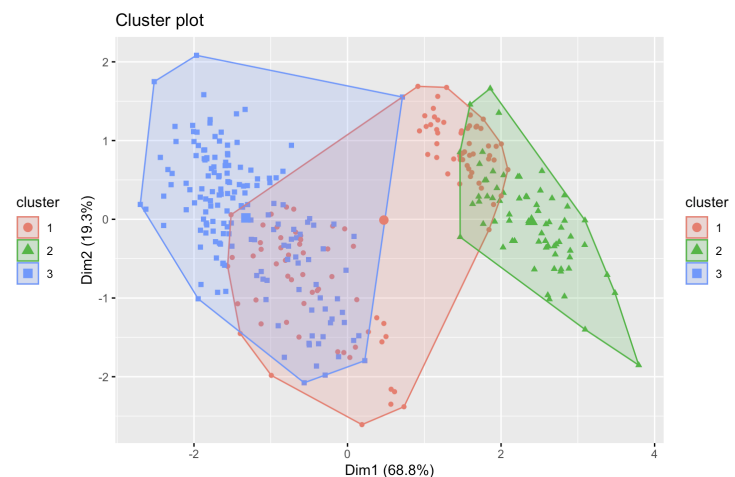
- Now, implement k -means clustering on the original (species-ignorant) dataset, `penguin_nospec` and use `fviz_cluster()` to visualise these results.
- Compare your two cluster visualisations, and comment on any differences.

```
# Q11
penguin_pca_km3 <- kmeans(penguin_pca$x[,1:2], centers=3, nstart=10)
fviz_cluster(penguin_pca_km3,
             data = penguin_pca$x[,1:2],
             labels = 0) # otherwise the observations are each labelled

# fviz_cluster(penguin_pca_km3,
#             data = penguin_nospec,
#             labels = 0) # otherwise the observations are each labelled

# Q12
penguin_km3 <- kmeans(penguin_nospec, centers=3, nstart = 10)
fviz_cluster(penguin_km3,
             data = penguin_nospec,
             labels = 0) # otherwise the observations are each labelled

# fviz_cluster(penguin_km3,
#             data = penguin_pca$x[,1:2],
#             labels = 0) # otherwise the observations are each labelled
```



The two panels above show the same projection of the underlying data; as mentioned above, `fviz_cluster()` uses PCA, as we have also done manually, to project the data onto a two-dimensional space. The difference is in the interpretation of the clusters shown. The left-hand panel shows the result of applying k -means clustering (with $k = 3$) to the projected data, whereas the right-hand panel shows the clusters, as calculated in the original, four-dimensional space. This is an important distinction to make, particularly when it comes to interpreting the clusters!

References

1. Horst AM, Hill AP, Gorman KB (2020). palmerpenguins: Palmer Archipelago (Antarctica) penguin data. R package version 0.1.0. Link here. (<https://allisonhorst.github.io/palmerpenguins/>)
2. Everitt, B, Landau, S, Leese, M, & Stahl, D (2011). *Cluster analysis*. (5th ed.) Wiley.
3. Silverman, B (1986). *Density Estimation*. Chapman and Hall.